

應用程式初始設定

來源：<https://codelabs.developers.google.com/app-onboarding?hl=zh-tw>

步驟數：6

目錄

1. [事前準備](#)
2. [準備工作區](#)
3. [使用預先定義和自訂的應用程式範本](#)
4. [建立應用程式的新執行個體](#)
5. [設定自動執行管道](#)
6. [自動完成所有新手上路步驟](#)

步驟 1 · 約 0 分鐘

1. 事前準備

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

The screenshot shows the Google Cloud Platform interface. At the top, there's a blue header with the Google Cloud Platform logo and a 'Select a project' button, which is circled in blue. Below this, there's a 'Select a project' section with a search bar. Further down, there's a 'New Project' section. In the 'New Project' section, the 'Project name' field is filled with 'my-kubernetes-codelab' and is circled in blue. Below the name field, the 'Project ID' is shown as 'my-kubernetes-codelab' with a note that it cannot be changed later. Below the ID field, the 'Location' is set to 'No organization'. At the bottom of the 'New Project' section, there are 'CREATE' and 'CANCEL' buttons.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串，您隨時可以更新。
- **專案 ID** 在所有 Google Cloud 專案中不得重複，且設定後即無法變更。Cloud 控制台會自動產生專屬字串，通常您不需要在意該字串為何。在大多數程式碼研究室中，您需要參照專案 ID (通常會標示為 `PROJECT_ID`)，因此如果您不喜歡該字串，可以產生另一個隨機字串，或是嘗試使用自己的字串，看看是否可用。專案建立後，系統就會「凍結」該值。
- 還有第三個值，也就是部分 API 使用的「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中不得重複，選定後即為專屬 ID，其他使用者無法使用。您是該 ID 的唯一使用者。專案刪除後，該 ID 就無法再使用。

注意：如果您使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果您使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成本程式碼研究室的費用應該不高，甚至完全免費。如要停用資源，避免在本教學課程結束後繼續產生帳單費用，請按照程式碼研究室結尾的「清除」操作說明

操作。Google Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

步驟 2 · 約 0 分鐘

2. 準備工作區

1. 前往下列網址開啟 Cloud Shell 編輯器

```
https://ide.cloud.google.com
```

2. 確認專案名稱已在 CLI 中設定

```
gcloud config set project {{project-id}}
```

```
export PROJECT_ID=$(gcloud config get-value project)
```

```
export PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --format='value(projectNumber)')
```

3. 啟用 API

```
gcloud services enable \
```

```
cloudbuild.googleapis.com \
```

```
secretmanager.googleapis.com
```

4. 提供 CloudDeploy 的權限

```
gcloud projects add-iam-policy-binding --  
member="serviceAccount:${PROJECT_NUMBER}@cloudbuild.gserviceaccount.com" --role  
roles/clouddeploy.admin ${PROJECT_ID}
```

```
gcloud projects add-iam-policy-binding --  
member="serviceAccount:${PROJECT_NUMBER}@cloudbuild.gserviceaccount.com" --role  
roles/container.developer ${PROJECT_ID}
```

```
gcloud projects add-iam-policy-binding --  
member="serviceAccount:${PROJECT_NUMBER}@cloudbuild.gserviceaccount.com" --role  
roles/iam.serviceAccountUser ${PROJECT_ID}
```

```
gcloud projects add-iam-policy-binding --  
member="serviceAccount:${PROJECT_NUMBER}@cloudbuild.gserviceaccount.com" --role  
roles/clouddeploy.jobRunner ${PROJECT_ID}
```

5. 在終端機視窗中，使用下列指令複製應用程式來源：

```
git clone https://github.com/GoogleCloudPlatform/software-delivery-workshop.git
```

6. 切換至該目錄，並將 IDE 工作區設為存放區根目錄

```
cd software-delivery-workshop && rm -rf .git
```

```
cd delivery-platform && cloudshell workspace .
```

3. 使用預先定義和自訂的應用程式範本

開發人員應可從機構內常用的範本中選擇。在導入程序中，系統會在您的 GitHub 帳戶中建立一組集中式範本存放區。在後續步驟中，這些範本存放區會經過複製和修改，做為新應用程式的基礎。在本實驗室中，您將使用這裡提供的範例結構，為範本存放區植入種子。您可以新增以範例為模型的其他資料夾，加入自己的範本。

在本步驟中，您將從提供的範例檔案建立自己的存放區，用來存放應用程式範本。系統提供輔助指令碼，簡化與 GitHub 的互動。

這些是一次性步驟，用於填入範本存放區。後續步驟會重複使用這些存放區。

設定 GitHub 存取權

本教學課程中的步驟會呼叫 GitHub API，以建立及設定存放區。在後續的各個階段，您都需要提供 GitHub 使用者名稱和個人存取權杖。下列指令碼可協助您取得值，並儲存為本機變數，以供日後使用。

```
source ./onboard-env.sh
```

```
echo Git Username: $GIT_USERNAME
```

```
echo Git Base URL: $GIT_BASE_URL
```

建立應用程式範本存放區

本實驗室提供範例應用程式範本，說明如何整合自己的基本範本。在這個步驟中，您會在 GitHub 帳戶中名為 `mcd-app-templates` 的存放區，建立這些檔案的副本。

1. 將範本複製到工作目錄

```
cp -R $BASE_DIR/resources/repos/app-templates $WORK_DIR
```

```
cd $WORK_DIR/app-templates
```

2. 在 GitHub 帳戶中建立空白的遠端存放區

```
$BASE_DIR/scripts/git/gh.sh create mcd-app-templates
```

3. 將範本存放區推送至遠端存放區

```
git init && git symbolic-ref HEAD refs/heads/main && git add . && git commit -m "initial commit"
```

```
git remote add origin $GIT_BASE_URL/mcd-app-templates
```

```
git push origin main
```

4. 清理工作目錄

```
cd $BASE_DIR
```

```
rm -rf $WORK_DIR/app-templates
```

建立共用基礎設定存放區

本教學課程會使用名為 `Kustomize` 的工具，該工具會使用多個團隊共用的基本設定檔，然後在頂端疊加應用程式專屬設定。方便平台團隊跨多個團隊和環境擴充。

在本步驟中，您會從提供的範例建立名為 `mcd-shared_kustomize` 的共用設定存放區

1. 將範本複製到工作目錄

```
cp -R $BASE_DIR/resources/repos/shared-kustomize $WORK_DIR
```

```
cd $WORK_DIR/shared-kustomize
```

2. 在 GitHub 帳戶中建立空白的遠端存放區

```
$BASE_DIR/scripts/git/gh.sh create mcd-shared_kustomize
```

3. 將範本存放區推送至遠端存放區

```
git init && git symbolic-ref HEAD refs/heads/main && git add . && git commit -m "initial commit"
```

```
git remote add origin $GIT_BASE_URL/mcd-shared_kustomize
```

```
git push origin main
```

4. 清理工作目錄

```
cd $BASE_DIR
```

```
rm -rf $WORK_DIR/shared-kustomize
```

建立範本存放區後，即可使用這些存放區建立應用程式例項

步驟 4 · 約 0 分鐘

4. 建立應用程式的新執行個體

從範本建立新應用程式時，通常需要將範本結構中多個檔案的預留位置變數換成實際值。完成替代作業後，系統會為新的應用程式例項建立新的存放區。開發人員會複製這個應用程式執行個體存放區，並在日常開發作業中使用。

在這個步驟中，您將替換應用程式範本中的值，並將產生的檔案發布至新的存放區。

定義新應用程式的名稱

```
export APP_NAME=my-app
```

擷取 Golang 範本存放區

```
cd $WORK_DIR/
```

```
git clone -b main $GIT_BASE_URL/mcd-app-templates app-templates
```

```
rm -rf app-templates/.git
```

```
cd app-templates/golang
```

替換預留位置值

在導入程序中，最常見的需求是將範本中的變數，換成應用程式中使用的實際例項。例如提供應用程式名稱。下列指令會使用環境變數中儲存的值，建立所有 .tmpl 檔案的執行個體。

```
for template in $(find . -name '*.tmpl'); do envsubst < ${template} > ${template%.*}; done
```

建立新的存放區並儲存更新後的檔案

1. 在 GitHub 帳戶中建立空白的遠端存放區

```
$BASE_DIR/scripts/git/gh.sh create ${APP_NAME}
```

2. 將範本存放區推送至遠端存放區

```
git init && git symbolic-ref HEAD refs/heads/main && git add . && git commit -m "initial commit"
```

```
git remote add origin $GIT_BASE_URL/${APP_NAME}
```

```
git push origin main
```

應用程式執行個體已建立完成，現在可以實作持續建構功能。

5. 設定自動執行管道

持續整合系統的核心功能，是根據來源控制系統中的事件執行管道邏輯。開發人員在存放區中提交程式碼時，系統會觸發事件，並可設定在其他系統中觸發程序。

在這個步驟中，您將設定 GitHub，以便在使用者於存放區中提交或標記程式碼時，呼叫 Google Cloud Build 並執行管道。

啟用安全存取

您需要 2 個元素，才能設定應用程式管道的安全存取權。管道專屬的 API 金鑰和密鑰。

API 金鑰

API 金鑰用於識別呼叫特定 API 的用戶端。在本例中，用戶端為 GitHub。這裡未提及的最佳做法是，將 API 金鑰的範圍限制為用戶端將存取的特定 API。您已在前一個步驟中建立金鑰。

1. 如要查看金鑰，請按一下[這個連結](#)
2. 執行下列指令，確保已設定值

```
echo ${API_KEY_VALUE}
```

管道密鑰

這些密鑰用於授權呼叫端，並確保呼叫端有權存取特定雲端建構目標工作。您在 GitHub 中可能有 2 個不同的存放區，但這些存放區只能存取自己的管道。API_KEY 會限制 GitHub 可使用的 API (在本例中為 Cloud Build API)，而密鑰則會限制用戶端可在 Cloud Build API 中執行的作業。

1. 定義密鑰名稱、位置和值

```
SECRET_NAME=${APP_NAME}-webhook-trigger-cd-secret
```

```
SECRET_PATH=projects/${PROJECT_NUMBER}/secrets/${SECRET_NAME}/versions/1
```

```
SECRET_VALUE=$(sed "s/[^a-zA-Z0-9]//g" <<< $(openssl rand -base64 15))
```

2. 建立密鑰

```
printf ${SECRET_VALUE} | gcloud secrets create ${SECRET_NAME} --data-file=-
```

3. 允許 Cloud Build 讀取密鑰

```
gcloud secrets add-iam-policy-binding ${SECRET_NAME} \
```

```
--member=serviceAccount:service-${PROJECT_NUMBER}@gcp-sa-cloudbuild.iam.gserviceaccount.com \
```

```
--role='roles/secretmanager.secretAccessor'
```

建立 Cloud Build 觸發條件

Cloud Build 觸發條件是實際執行 CICD 程序的設定。

建立工作時，您必須提供幾個重要值，才能正確設定觸發條件。

4. 定義觸發條件的名稱，以及設定檔所在位置

```
export TRIGGER_NAME=${APP_NAME}-cloudeploy-webhook-trigger
```

```
export BUILD_YAML_PATH=$WORK_DIR/app-templates/golang/build/cloudbuild-cd.yaml
```

5. 定義共用基本設定存放區的位置。

```
export KUSTOMIZE_REPO=${GIT_BASE_URL}/mcd-shared_kustomize
```

6. 在 onboard-env.sh 指令碼中設定的變數，會定義專案的 Container Registry。使用下列指令檢查值。

```
echo $IMAGE_REPO
```

7. 使用先前建立的變數，建立 CloudBuild Webhook 觸發條件。應用程式存放區位置是從 GitHub 要求的內文中提取。下方的值是指要求內容中的路徑，該路徑位於 `gcloud alpha builds triggers create webhook \`

```
`--name=${TRIGGER_NAME} \`

`--
substitutions='_APP_NAME='${APP_NAME}',_APP_REPO=$(body.repository.git_url),_CONFIG_REPO=
'${GIT_BASE_URL}'/'${CLUSTER_CONFIG_REPO}',_DEFAULT_IMAGE_REPO='${IMAGE_REPO}',_KUSTOMIZE
_REPO='${GIT_BASE_URL}'/'${SHARED_KUSTOMIZE_REPO}',_REF=$(body.ref)' \`

`--inline-config=$BUILD_YAML_PATH \`

`--secret=${SECRET_PATH}`
```

8. [按一下這個連結](#)，在控制台中查看新建立的 Cloud Build 觸發程序

9. 定義端點網址的變數，GitHub 會在下一個步驟中使用

```
WEBHOOK_URL="https://cloudbuild.googleapis.com/v1/projects/${PROJECT_ID}/triggers/${TRIGGER_NAME}
:webhook?key=${API_KEY_VALUE}&secret=${SECRET_VALUE}"
```

設定 GitHub Webhook

1. 在 GitHub 中設定 Webhook

```
$BASE_DIR/scripts/git/gh.sh create_webhook ${APP_NAME} $WEBHOOK_URL
```

2. 前往應用程式存放區，檢查新設定的 Webhook

```
REPO_URL=${GIT_BASE_URL}/${APP_NAME}/settings/hooks
```

```
echo $REPO_URL
```

您已手動執行建立新應用程式所需的所有步驟，現在可以開始在指令碼中自動執行這些步驟。

6. 自動完成所有新手上路步驟

實際上，為每個新應用程式執行上述步驟並不可行。而是應將邏輯併入指令碼，方便執行。上述步驟已納入供您使用的指令碼。

在這個步驟中，您將使用提供的指令碼建立新應用程式

建立新應用程式

1. 確認您位於正確的目錄

```
cd $BASE_DIR
```

2. 建立新應用程式

```
export APP_NAME=demo-app
```

```
./app.sh create ${APP_NAME}
```

所有步驟都會自動執行。

查看 GitHub 存放區

此時，您可以在 GitHub 中查看新存放區

1. 執行下列指令，擷取 GitHub 存放區網址

```
echo ${GIT_BASE_URL}/demo-app
```

2. 使用網路瀏覽器開啟網址，即可查看新應用程式
3. 請注意範例，其中範本變數已替換為執行個體值，如下方網址所示

```
echo ${GIT_BASE_URL}/demo-app/blob/main/k8s/prod/deployment.yaml#L24
```

4. 查看在下方網址設定的 Webhook

```
echo ${GIT_BASE_URL}/demo-app/settings/hooks
```

查看 Cloud Build 觸發條件

觸發條件是由指令碼自動設定

1. [按一下這個連結](#)，在控制台中查看 Cloud Build 觸發條件
 2. 查看[這個頁面](#)的建構記錄
-